

Open the evidence document, **evidence.doc**

Make sure that your name, centre number and candidate number appear on every page of this document. This document must contain your answers to each question.

Save this evidence document in your work area as:

evidence_ followed by your centre number_candidate number, for example: **evidence_zz999_9999**

A class declaration can be used to declare a record. If the programming language used does **not** support arrays, a list can be used instead.

One source file is used to answer **Question 3**. The file is called **HighScoreTable.txt**

- 1** A computer game is designed for users to select characters. Each character can take part in a group of events. Each group has five events. There are four types of event: jump, swim, run, drive.

The program is written using object-oriented programming.

- (a)** The class `EventItem` stores data about the events.

EventItem	
EventName : STRING	stores the name of the event
Type : STRING	stores the type of event, either: jump, swim, run or drive
Difficulty : INTEGER	stores the difficulty of the event from 1 (easiest) to 5 (hardest)
Constructor()	initialises EventName, Type and Difficulty to its parameter values
GetName()	returns the name of the event
GetDifficulty()	returns the difficulty of the event
GetEventType()	returns the type of event

- (i)** Write program code to declare the class `EventItem` and its constructor.

Do **not** declare the other methods.

Use your programming language's appropriate constructor.

All attributes must be private.

If you are writing in Python, include attribute declarations, using comments.

Save your program as **Question1_N24**.

Copy and paste the program code into part **1(a)(i)** in the evidence document.

[4]

- (ii) The get methods `GetName()`, `GetDifficulty()` and `GetEventType()` each return the relevant attribute.

Write program code for the **three** get methods.

Save your program.

Copy and paste the program code into part **1(a)(ii)** in the evidence document.

[3]

- (b) The array `Group` stores five objects of type `EventItem`.

- (i) Write program code to declare `Group` local to the main program.

Save your program.

Copy and paste the program code into part **1(b)(i)** in the evidence document.

[1]

- (ii) One group has the following events:

Event name	Event type	Event difficulty
Bridge	jump	3
Water wade	swim	4
100 mile run	run	5
Gridlock	drive	2
Wall on wall	jump	4

Write program code to create an instance of `EventItem` for each of the **five** events, and store them in `Group`.

Save your program.

Copy and paste the program code into part **1(b)(ii)** in the evidence document.

[3]

(c) The class `Character` stores data about the characters in the game.

Each character has a skill level for each type of event. The skill level is an integer between 1 and 5 inclusive. Skill level 1 is the lowest skill level, and skill level 5 is the highest skill level.

Character	
<code>CharacterName : STRING</code>	stores the name of the character
<code>Jump : INTEGER</code>	stores the character's skill level at events of type jump
<code>Swim : INTEGER</code>	stores the character's skill level at events of type swim
<code>Run : INTEGER</code>	stores the character's skill level at events of type run
<code>Drive : INTEGER</code>	stores the character's skill level at events of type drive
<code>Constructor()</code>	initialises <code>CharacterName</code> , <code>Jump</code> , <code>Swim</code> , <code>Run</code> and <code>Drive</code> to its parameter values
<code>GetName()</code>	returns the name of the character
<code>CalculateScore()</code>	takes the type of event and difficulty as parameters. Calculates and returns the chance of the character completing the event

Write program code to declare the class `Character`, its constructor and get method.

Use your programming language's appropriate constructor.

All attributes must be private.

If you are writing in Python, include attribute declarations, using comments.

Save your program.

Copy and paste the program code into part **1(c)** in the evidence document.

[4]

- (d) The method `CalculateScore()` in the `Character` class calculates and returns the percentage chance of a character completing an event.

When a character's skill level is greater than or equal to the difficulty of that event, the percentage chance of completing the event is 100%.

When a character's skill level is less than the difficulty of that event, the character's skill level is subtracted from the difficulty of that event. This difference is used to identify the percentage chance of success using this table:

Difference	Percentage chance of success
1	80
2	60
3	40
4	20

For example:

- A character has a skill level of 3 for events of type run.
- An event of type run has a difficulty level of 5
- The character's skill level is less than the difficulty, therefore the difference is calculated.
- The difference is the character's skill level subtracted from the event difficulty, $5 - 3 = 2$
- The difference is 2, therefore the percentage chance of succeeding is 60%

Write program code for the method `CalculateScore()` to:

- take the type of event and difficulty as parameters
- calculate the percentage chance of the character completing the event
- return the percentage chance of completing the event as an integer number, for example 60

Save your program.

Copy and paste the program code into part **1(d)** in the evidence document.

[4]

- (e) Two characters are attempting each event in the group you created in part 1(b).

One character has the name Tarz and the skill levels:

Jump 5

Swim 3

Run 5

Drive 1

The second character has the name Geni and the skill levels:

Jump 2

Swim 2

Run 3

Drive 4

Each `Character` object is stored in a variable.

- (i) Amend the main program to declare and create an instance of a `Character` object for Tarz and for Geni.

Save your program.

Copy and paste the program code into part 1(e)(i) in the evidence document.

[2]

- (ii) Both characters Tarz and Geni take part in each event in the group you created in part 1(b).

These steps are repeated for all **five** events:

- The percentage chance of each character completing the event is calculated and compared.
- The character with the highest percentage chance of completing the event gets 1 point. Their character name is output together with a message telling them they have won that event.
- If both characters have the same percentage chance of completing the event, the scores do **not** change, and a message is output stating that the event is a draw.

When the total score for each character has been calculated, the name of the character with the highest score is output stating they have won and the number of points they have. If both characters have the same number of points, a message is output telling them the group is a draw.

Amend the main program to:

- calculate the score for each character in each event
- output the name of the character that wins each event or that the event is a draw
- output the name of the character that has the most points for the group or that the group is a draw.

Save your program.

Copy and paste the program code into part 1(e)(ii) in the evidence document.

[7]

- (iii) Test your program.

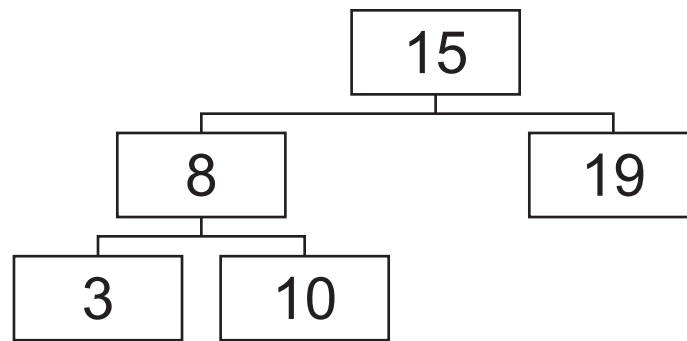
Take a screenshot of the output.

Save your program.

Copy and paste the screenshot into part 1(e)(iii) in the evidence document.

[1]

- 2 A binary tree stores data in ascending order. For example:



A computer program stores integers in a binary tree in ascending order. The program uses Object-Oriented Programming (OOP).

The binary tree is stored as a 1D array of nodes. Each node contains a left pointer, a data value and a right pointer.

The class `Node` stores the data about a node.

Node	
<code>LeftPointer : INTEGER</code>	stores the index of the node to the left in the binary tree
<code>Data : INTEGER</code>	stores the node's data
<code>RightPointer : INTEGER</code>	stores the index of the node to the right in the binary tree
<code>Constructor()</code>	initialises <code>Data</code> to its parameter value initialises <code>LeftPointer</code> and <code>RightPointer</code> to <code>-1</code>
<code>GetLeft()</code>	returns the left pointer
<code>GetRight()</code>	returns the right pointer
<code>GetData()</code>	returns the data value
<code>SetLeft()</code>	assigns the parameter to the left pointer
<code>SetRight()</code>	assigns the parameter to the right pointer
<code>SetData()</code>	assigns the parameter to the data

- (a) (i) Write program code to declare the class `Node` and its constructor.

Do **not** declare the other methods.

Use the appropriate constructor for your programming language.

If you are writing in Python, include attribute declarations using comments.

Save your program as **Question2_J24**.

Copy and paste the program code into part **2(a)(i)** in the evidence document.

- (ii) The get methods `GetLeft()`, `GetRight()` and `GetData()` each return the relevant attribute.

Write program code for the **three** get methods.

Save your program.

Copy and paste the program code into part **2(a)(ii)** in the evidence document.

[3]

- (iii) The set methods `SetLeft()`, `SetRight()` and `SetData()` each take a parameter and then store this in the relevant attribute.

Write program code for the **three** set methods.

Save your program.

Copy and paste the program code into part **2(a)(iii)** in the evidence document.

[3]

- (b) The class `TreeClass` stores the data about the binary tree.

TreeClass	
<code>Tree[0:19] : Node</code>	an array of 20 elements of type <code>Node</code>
<code>FirstNode : INTEGER</code>	stores the index of the first node in the tree
<code>NumberNodes : INTEGER</code>	stores the quantity of nodes in the tree
<code>Constructor()</code>	initialises <code>FirstNode</code> to <code>-1</code> and <code>NumberNodes</code> to <code>0</code> initialises each element in <code>Tree</code> to a <code>Node</code> object with the data value of <code>-1</code>
<code>InsertNode()</code>	takes a <code>Node</code> object as a parameter, inserts it in the array and updates the pointer for its parent node
<code>OutputTree()</code>	outputs the left pointer, data and right pointer of each node in <code>Tree</code>

Nodes cannot be deleted from this tree.

- (i) Write program code to declare the class `TreeClass` and its constructor.

Do **not** declare the other methods.

Use the appropriate constructor for your programming language.

If you are writing in Python, include attribute declarations using comments.

Save your program.

Copy and paste the program code into part **2(b)(i)** in the evidence document.

[4]

- (ii) The method `InsertNode()` takes a `Node` object, `NewNode`, as a parameter and inserts it into the array `Tree`.

`InsertNode()` first checks if the tree is empty.

If the tree is empty, `InsertNode()`:

- stores `NewNode` in the array `Tree` at index `NumberNodes`
- increments `NumberNodes`
- stores 0 in `FirstNode`.

If the tree is **not** empty, `InsertNode()`:

- stores `NewNode` in the array `Tree` at index `NumberNodes`
- accesses the data in the array `Tree` at index `FirstNode` and compares it to the data in `NewNode`
- repeatedly follows the pointers until the correct position for `NewNode` is found
- once the position is found, `InsertNode()` sets the left or right pointer of its parent node
- increments `NumberNodes`.

Write program code for `InsertNode()`.

Save your program.

Copy and paste the program code into part **2(b)(ii)** in the evidence document.

[6]

- (iii) The method `OutputTree()` outputs the left pointer, the data and the right pointer for each node that has been inserted into the tree. The outputs are in the order they are saved in the array.

If there are no nodes in the array, the procedure outputs 'No nodes'.

Write program code for `OutputTree()`.

Save your program.

Copy and paste the program code into part **2(b)(iii)** in the evidence document.

[4]

- (c) (i) The main program declares an instance of `TreeClass` with the identifier `TheTree`.

Write program code for the main program.

Save your program.

Copy and paste the program code into part **2(c)(i)** in the evidence document.

[1]

- (ii) The main program inserts the following integers into the binary tree in the order given:

10

11

5

1

20

7

15

The main program then calls the method `OutputTree()`.

Write program code to amend the main program.

Save your program.

Copy and paste the program code into part **2(c)(ii)** in the evidence document.

[4]

- (iii) Test your program.

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **2(c)(iii)** in the evidence document.

[1]